UF Herbert Wertheim College of Engineering
UNIVERSITY of FLORIDA

EEL4712 Digital Design

Lecture 15 - VGA Pixel Generation with Memories

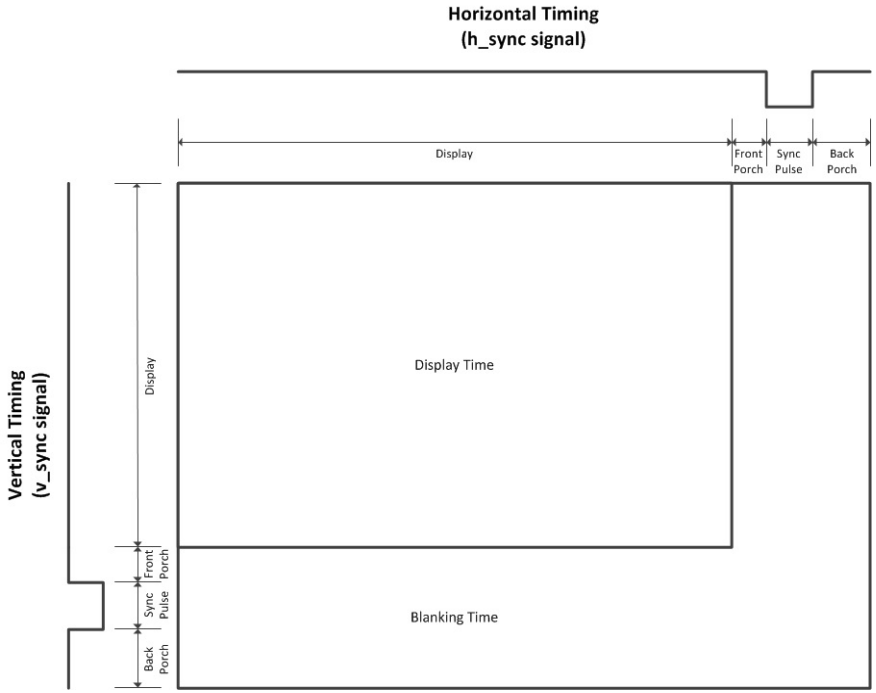
Rickard Ewetz
Spring 2026

Agenda

1. Review of Previous Lecture
2. Memory as a Pixel Source
3. Pixel Generation Architecture

1. Review of Previous Lecture

- h/v counters
- Sync pulses
- display_on
- Pixel coordinate pipeline



2. Memory as a Pixel Source

- ROM for patterns
- BRAM for sprites
- Large memories → no full framebuffer (too big)
  - 640 × 480 × 12 bits ≈ **3.7 Mbits**
  - Most teaching FPGAs have **~1–2 Mbits total BRAM**
  - Your boards have 64MB SDRAM but more complicated to use. It is not random access.

- Memory bandwidth requirement: one pixel per clock

Use Generated Memory Using MegaWizard Plug-In Manager

- Purpose of MIF (preload patterns or sprites
- Supported in Quartus and various tools
- Equivalent COE file for Vivado

Example MIF structure:

```
WIDTH=8;
DEPTH=256;
ADDRESS_RADIX=HEX;
DATA_RADIX=HEX;
CONTENT BEGIN
00 : FF;
01 : 2A;
...
END;
```

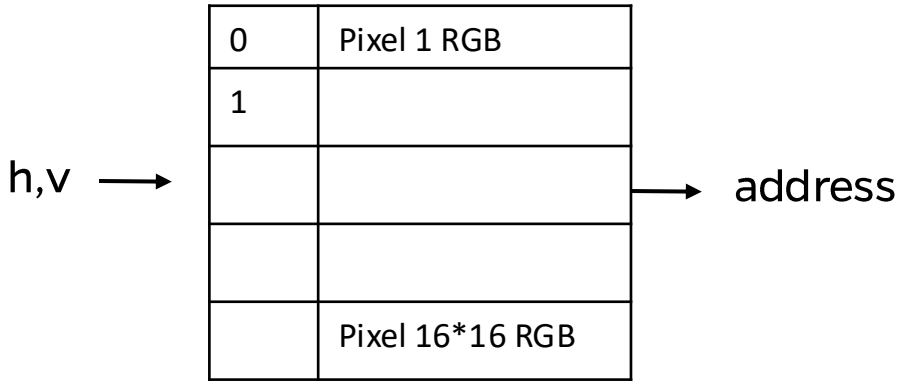
5. Pixel Generator Architecture

- Inputs: h, v, display_on
- Use (h, v) to address a memory **address**
- Example Problems:
  - Full screen pattern?
  - 16×16 tile pattern repeated
  - Or a 16x16 sprite drawn at specific (x_pos, y_pos)

Sprite 4x4

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Given (h,v), how do we get the corresponding memory address?



Address Generation Examples

- Example 1 — Full-pattern (N×M):
  - addr = v * WIDTH + h
- Example 2 — Repeating tile:
  - addr = (v mod 16) * 16 + (h mod 16)
- Example 3 — Sprite window:
  - if h in [x_pos, x_pos+15] and v in [y_pos, y_pos+15] then
    sprite_addr <= (v - y_pos)*16 + (h - x_pos);

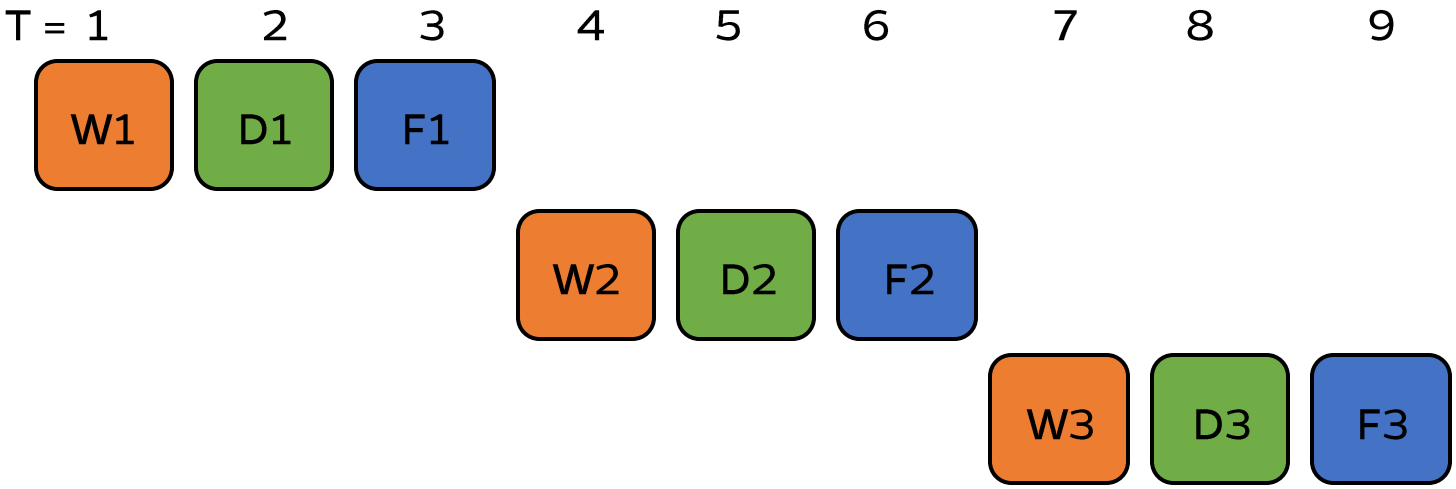
7. Synchronous BRAM Timing

- BRAM outputs pixel one clock after address is applied
- Need to pipeline:
  - display_on
  - (h, v)
  - Pixel data

What is Pipelining?

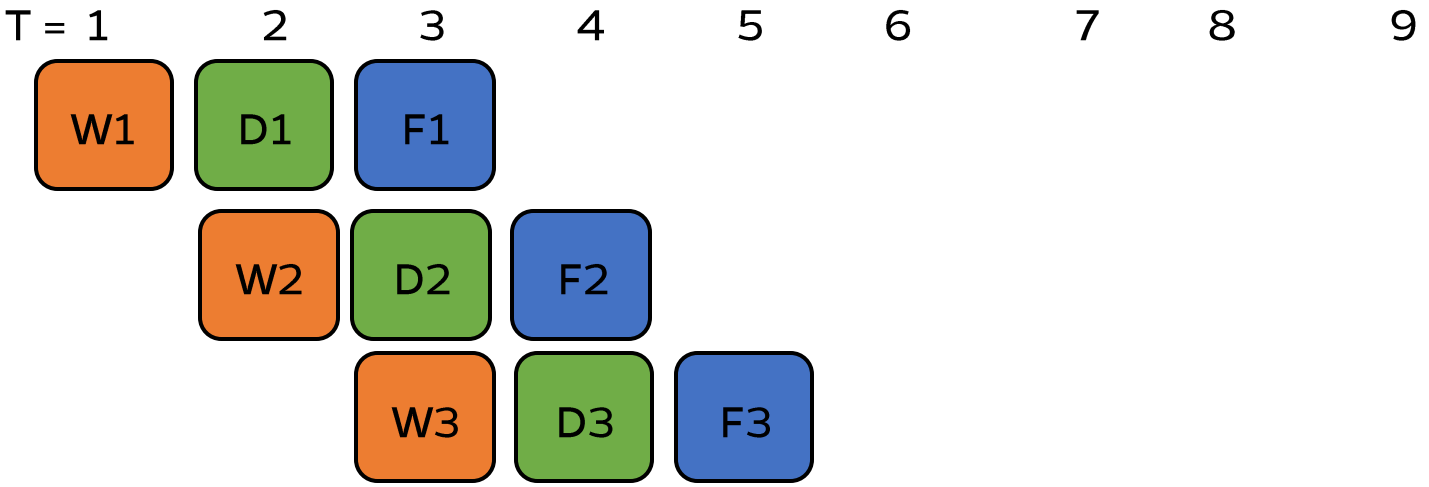
- Laundry pipeline
- Without pipelining:
  - Wash
  - Dry
  - Fold
- Total time for 3 loads = long.

- With pipelining:
  - Load 1 drying
  - Load 2 washing
  - Load 3 ready
- Throughput increases



What is Pipelining?

- With pipelining:
  - Load 1 drying
  - Load 2 washing
  - Load 3 ready
- Throughput increases



Digital Example

- Suppose we compute:

$$Y = (A + B) \times C$$

- Without pipelining
  - Clock cycle 1: add
  - Clock cycle 2: multiply
  - Clock cycle 3: output
- One result every **3 cycles**

- With Pipelining
- Break into stages:
  - Stage 1: add
  - Stage 2: multiply
  - Stage 3: register output
- Now:
  - Cycle 1 → compute add (1)
  - Cycle 2 → multiply (1)
  - Cycle 2 → add (2)
  - Cycle 3 → output (1)
  - Cycle 3→ add (3)
  - Cycle 3→ multiply (2)
- Then: one result every cycle.

Why Pipelines Need Registers

- Registers:
  - separate stages
  - store intermediate results
  - allow each stage to run every clock
- Without registers → not pipelined.

Advantages vs. Disadvantages

- Benefits of Pipelining
  - increases throughput
  - enables high clock frequency
  - improves performance
  - enables continuous data flow
- Costs of Pipelining
  - increased latency
  - more registers
  - control complexity
  - alignment challenges

8. Building the Pixel Pipeline

- Pixel Pipeline
  - Stage 1: compute address
  - Stage 2: BRAM read
  - Stage 3: apply RGB output
- Align pipeline to avoid shifted images

9. Integrating Timing Generator + Pixel Generator

- Instantiate timing module
- Instantiate pixel module
- Wire counters into pixel generator
- Combine outputs into VGA Digital to Analog Converter (DAC) pins

DRAW BLOCK
DIAGRAM in class

Details of Lab 5

- Display
  - 2x2 pixel-sized blocks
  - 4096 blocks arranged as a 64x64 grid
  - 128x128 image
  - VGA resolution is 640x480
  - Switch/button location
- BROM contains RGB values for each 64x64 grid.
  - Size of ROM is 4096x12
  - 12-bit address (2^12 = 4096) and a 12-bit output
  - Form BROM address from block row and column

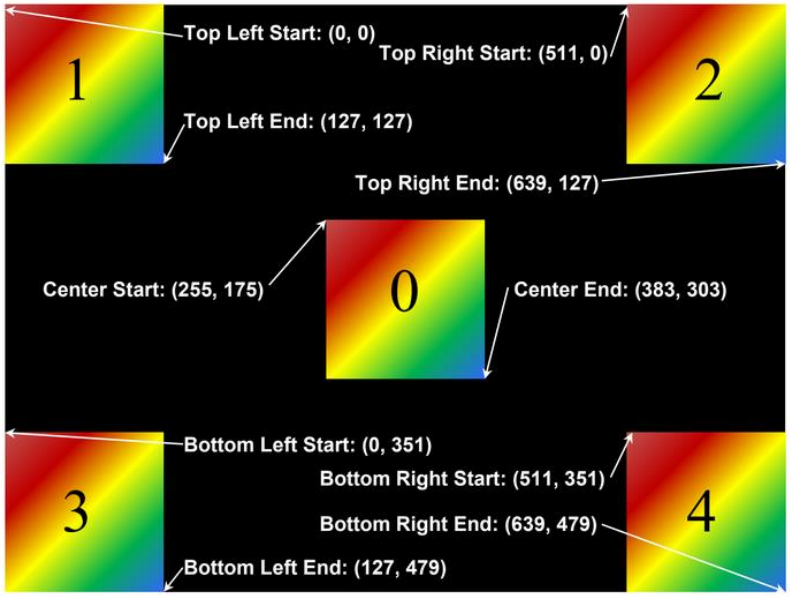


Figure 4. Display positions for different button combinations. Exactly one of these can be active at a time.

Pixel 1: 12 bits (RGB)
Pixel 2: 12 bits (RGB)
Pixel: 4096

Block Address Logic

- Block row address logic
  - Vcount signal and generates a 6-bit row address that identifies one of the 64 rows of the 64x64 grid.
  - Consider button
  - Enable output
- Block column address logic
  - Hcount signal and generates a 6-bit column address that identifies one of the 64 columns of the 64x64 grid.
  - Consider button
  - Enable output
- BROM ROM address
  - A = hBlock + vBlock*W

FSMD vs FSM+D

- FSM+D has control and datapath in separate files
- FSMD has both control and datapath in the same VHDL file that is implemented using X-process model
  - FSMD can be used for describing circuits of low to medium complexity
  - For complex designs, FSM+D is typically the best choice
  - You can implement the block column address logic using FSMD or FSM+D

10. FSMD Animation

- Sprite movement every frame
- Boundary detection
- FSMD updates state variables
  - What are the state variables?

How to implement this? What is the FSM and Datapath?

Extra Credit Opportunity: Earn 0.5% added to your course grade by implementing an animation in which the sprite moves and bounces off the screen edges (left, right, top, and bottom).

11. Debugging VGA Memory Systems

- All-black screen → forgot MIF or display_on
- Wrong colors → incorrect memory width
- Sprite shifted → forgot BRAM latency
- Monitor complains → wrong sync timing

12. Summary

- Using ROM/BRAM to store pixel data
- MIF file initialization
- Address generation
- Pipelined pixel generator
- Integrating pixel generator + timing generator
- FSMD-driven animation